Development Plan
Software Engineering

Team 20, Aeolus Software
Andy Huynh
Andreas Sideris
Humna Saeed
Christopher Ludwig
Owen Johnson

Table 1: Revision History

Date	**Developer(s)**	**Change**
September 22, 2026	Andy	added rough draft for PoC and Expected Tech
September 23, 2026	Andreas Sideris	Initial draft of Workflow Plan
September 23, 2026	Andy	added revisions to PoC and Expected Tech
September 24, 2026	Andreas Sideris	Initial draft of Project Decomposition and Scheduling
September 25, 2026	Andreas Sideris	Use of AI and LLMs Completed
September 26, 2026	Owen Johnson	Confidential Information and IP to Protect Completed
September 26, 2026	Christopher Ludwig	Team Charter, Communication, Meetings, Roles Rough Draft
September 26, 2026	Andreas Sideris	Coding Standard Added
September 27, 2026	Owen Johnson	PoC Demo Plan Expanded, Expected Technology Slightly Edited
September 27, 2026	Humna Saeed	Introduction, Team Charter
September 27, 2026	Andreas Sideris	Added Reflection
September 27, 2026	Owen Johnson	PoC Demo Plan Rewritten, Added Reflection
September 28, 2026	Owen Johnson	PoC Demo Plan Edited, Added Copyright License
September 28, 2026	Humna Saeed	Added reflection
September 28, 2026	Christopher Ludwig	Fixed formatting and cleaned up contents of Team Meeting Plan, Communication Plan, and Team Roles, as well as adding reflection

The following development plan covers critical areas that will aid throughout the development of RealmForge, including core project management decisions, team organization, project scheduling, workflow, proof of concept, and expected technologies.

1 Confidential Information?

Protected Information: This project does not have any confidential information to protect.

2 IP to Protect

Project Type: Internal Capstone

Based on the Capstone Projects IP Guidelines document in the course repository, this project is an internal capstone. This means that the students own the IP by default, no agreement or acknowledgement is required, and MILO does not need to be involved.

3 Copyright License

Copyright License Type: Creative Commons Attribution ShareAlike 4.0.

- This license allows distributing, remixing, adapting, and building upon your work
- Work must only be used for non-commercial purposes
- Attribution must be given to creator(s)
- This license allows distributing, remixing, adapting, and building upon your work
- Use of this work must be licensed under the same license (or a compatible one)

The License File is found in the project repository.

4 Team Meeting Plan

Meetings will be held every Monday from 2:30pm - 3:30pm. More meetings will be scheduled if needed, prioritizing lecture/tutorial time as everyone should be available then. Each meeting will be held according to the following standards:

- Each meeting will have a corresponding issue created, containing a previously made agenda, all decisions made, tasks assigned, deadlines and next steps/action items and anything else discussed

- In case of member being unable to attend, they must Join a Microsoft Teams meeting to participate virtually if possible. If completely busy at that time, they must inform the rest of the team 1 day prior if possible, as well as reviewing the agenda and providing any information relevant to the meeting, and then review any documents produced after
- Roles will be assigned to team members to keep meetings structured and important items and tasks will not be missed

5 Team Communication Plan

There are many different channels of communication our team will employ. Communication channels include but are not limited to:

- Primarily Microsoft Teams, split up into different channels to keep discussion organized
- Meetings and informal conversation between team members
- GitHub Issues

Any important information and decisions pertaining to the project should be recorded under the related GitHub issue or have a new one created if one does not exist already.

6 Team Member Roles

Roles are broken down into 3 sections: Meeting roles, Technical roles, and Research roles. Each team member should embrace all roles present, not leaving the responsibility of each of these roles entirely on one person, however their main focus should be on the roles assigned to them. Roles are subject to change in the future as well as being reassigned if the team sees fit.

Meeting Roles:

- Chair - Generally runs meetings, taking lead on agenda creation and keeping the team focused to cover all action items during meeting - Andreas
- Scribe - Records everything discussed, updating tasks, deadlines, and next steps/action items on the GitHub Repository - Andy
- Progress Tracker - Follows up on details from previous meetings and deadlines to ensure team stays on track with older tasks - Chris
- Quality/Scope Assessor - Assesses the scope of each decision made and ensures that team stays within the desired workload - Humna

- Risk Assessor - Should question all assumptions made and attempt to poke holes in all decisions, should try and get team to think deeply about everything - Owen

Technical Roles:

- Systems Architect: Takes lead on architecture design and oversees development is according to the blueprint - Andy
- Back-end Lead: Leads development of the back-end of the program - Chris
- Front-end Lead: Leads development of the front-end of the program - Humna, Owen
- CI/CD Engineer: Focuses on creating workflows to support CI and CD - Andreas

Research Roles:

- Research Design Lead: Formulates the research questions, hypotheses, and experimental methodology for evaluating the user experience. - Chris
- Ethics and Participant Coordinator: Drafts consent forms and helps recruit study participants - Andy
- Data Collection Analyst: Builds logging mechanisms into the software to capture quantitative user metrics (time-played, score, etc.) - Owen
- Qualitative Research Lead: Conducts/designs post-test interviews, surveys and focus groups - Andreas
- Statistical Data Analyst: Analyzes quantitative data to drive further decisions - Humna

7 Workflow Plan

Git Development Workflow: The project will be version controlled through the use of a GitHub repository, with reviewed and tested source code and documentation stored in the 'main' branch. The project's files will be managed using the Feature Branch Git Workflow. The complete planned workflow is described below. When working on source code or documentation, team members will:

- Start from the main branch, and ensure their local copy is up to date.
- Create a new Git feature branch to push their work to.
- Complete their work, and regularly stage and commit changes to the feature branch.
- Push their changes to the feature branch.

- Create a pull request to merge their changes with the main branch.
- Request fellow team members to review their pull request.
- Merge the pull request with the main branch, once other team members have completed their review and approved the request.

Branches: Branches should have descriptive names, related to the work being done. They should be written with lower case text, and multiple words should be hyphenated.

Pull Requests: Pull Requests (PRs) should follow the guidelines below:

- PRs should have descriptive names, detailing the feature or document section being updated, and what was changed.
- PRs should have detailed descriptions, explaining the work that was done in their changes, and what must be reviewed.
- PRs should be linked to GitHub issues that are related to their changes, such as the feature(s) or documentation section(s) that were completed through the changes.
- PRs should state the reviewer(s) that are planned to approve them. Reviewers can be previously planned when beginning work on the issue, or selected when the changes are planned to be merged (after discussion with the team). Aside from these specific reviewers, all team members are permitted to provide feedback on PRs, if necessary.
- Team members that created a PR and worked on its content, as well as planned reviewers, should be assigned to the PR on GitHub.

Issues: Issues should follow the guidelines below:

- Issues should have descriptive, title case names, detailing the work product involved (e.g. 'Feature A'). If needed, issue names can also contain descriptions of what will be done to complete them (e.g. 'Build System B', or 'Fix System C Bug'). For documentation, issues pertaining to a specific document section should be named using the format 'Document: Section'.
- Issues should have detailed descriptions, clearly stating the work required to complete each issue.
- Issues should have their task/issue owner assigned to them on GitHub.
- Sub issues will not be used, to minimize unneeded overhead.
- Documentation focused issues should use the Documentation issue template.

- Source code feature focused issues should use the Feature issue template.
- Bug fix focused issues should use the Bug issue template.
- Issues may use other labels and priorities as needed.
- Issues should be linked to relevant Projects and Milestones.

Checklists: Provided deliverable checklists should be reviewed by all team members before any required course deliverables are submitted.

Milestones: Milestones should be created for each major course deliverable. They should have a descriptive name, and should be linked to the issues required to complete the deliverable.

CI/CD: CI/CD pipelines will be used in the project's repository, through GitHub Actions. At a baseline level, CI/CD will be used for automated tests of source code, merged into the main project repository branch. Use of CI/CD pipelines may be expanded to pull request testing, code linting, or other purposes if needed (and if time is available for their construction).

8 Project Decomposition and Scheduling

GitHub projects will be used to keep track of upcoming course/project deliverables (as milestones), and the status of work tasks (as issues) linked to those deliverables. The GitHub project used for our team's repository is linked here: RealmForge Project Tracker. This project contains a primary issue backlog table view, with fields displaying each issue's task owner (assignee), its status ('Todo', 'In progress', 'In review', or 'Done'), any linked pull requests, the milestone the issue contributes towards, its type (Task, Feature, or Bug), and its target date (matching the deadline of the milestone it is linked to).

When work begins for an upcoming milestone, all team members will collaborate in recurring team meetings to divide the milestone into suitable work tasks, which will be created as issues and linked to the GitHub project. This will involve splitting documentation by different sections, and subjects. For source code, development will be split across natural demarcations (e.g. split into 'Feature A' and 'Feature B', or 'Test System C' and 'Test System D'). Reviewers will be selected for each issue, and each team member will complete their assigned tasks, collaborating with other members as needed. The division of work will be reassessed for a given milestone if any team members suggest that they unable to complete their work without assistance.

The initial project schedule below corresponds to GitHub milestones created and linked to the GitHub project mentioned above. This schedule may be revised if any deliverable logistics change, or if the team chooses to alter it following more detailed planning (i.e. pushing due dates earlier to prepare earlier,

etc.).

Initial project milestone schedule:

- M1 Problem Statement, Development Plan, PoC Plan, and Team Charter (Due by September 28, 2026)
- M2 SRS and Hazard Analysis Revision 0 (Due by October 19, 2026)
- M3 VnV Plan Revision 0 (Due by November 2, 2026)
- M4 Design Document Revision -1 and CD-Fall (FODA) (Due by November 16, 2026)
- M5 PoC Demo (Due by November 23, 2026)
- M6 Design Documentation Revision 0 (Due by January 18, 2027)
- M7 Revision 0 Demo (Due by February 1, 2027)
- M8 VnV Report Revision 0 and CD-Winter Revision 0 (Usability Report) (Due by March 8, 2027)
- M9 Final Demonstration Revision 1, EXPO Demonstration (Due by March 15, 2027)
- M10 Final Documentation (Revision 1) (Due by March 31, 2027)

9 Proof of Concept Demonstration Plan

The main risks identified are an initial **Learning Curve**, the possibility of **Scope Creep**, and the resulting implementation being **Hard to Modify/Vary**.

With whatever expected technology the team will use, there will certainly be a learning curve for the game engine/framework chosen. This could prove to be a barrier when creating complex portions of the project, and can make it hard to write robust code straight away since this is new software.

Scope creep is an important risk to acknowledge as well. With today's technologies such as AI and LLMs it is much easier to implement new features without thinking it all the way through. If scope creep becomes an issue it may become more difficult to implement new features, since building features without solidifying the core components of the project leads to them being more complex and taking more time to implement.

The aforementioned risks culminate into a weak code base, which could be less modifiable and extendable than the project demands it to be. This would make implementing the myriad of feature variation points a very difficult task to the

point of not being able to meet deadlines.

To prove these risks can be overcome, the proof of concept demonstration will include strong core components. To avoid scope creep it will stick to these core components so that they are very robust and extendable by the time they are finished. This means:

- Basic unit testing with good code coverage
- The main encounter/combat loop, with variation points for features:
  - At least 1 distinct enemy, with varying properties
  - At least 2 Player/Enemy properties
  - Central Hub operations to gather resources and upgrade production for at least 2 resources
  - At least 1 item/skill that uniquely affects Player/Enemy/Hub properties when used
  - Encounter completion conditions and reward options (different between runs)

It is imperative that important gameplay features have elements of **modifiability** and **scalability**. This will ensure that the features are robust and that the current **feature variation points** set a good precedent for implementing further variability. This variability is important, as it forms the basis of the subsequent usability testing.

In the event of a failed proof of concept, the team has various methods of regrouping. First, the scope of the project could be pared down, focusing on more rigid plans and deadlines for core features. This would aid the direction of the project and ensure that it stays on course, avoiding scope creep through adding unnecessary features. Another method to refocus could be pivoting some key structure of the game to make it easier to fully flesh out. For example, changing the game engine/framework to something more familiar would allow the team to worry more about the quality of the work rather than learning while developing features.

10 Expected Technology

There are a number of game engines and frameworks that vary between ease of use, flexibility, cross-platform availability and other capabilities. The team has narrowed it down to mainly Godot, GameMaker, Unity and LibGdx based on their respective pros and cons, although more programming technologies can be postulated if those prove not-suitable to the task. If the project uses Godot or GameMaker then the team follows the engine's respective language GDScript or GameMakerLanguage(GML), similarly if the project uses Unity or LibGDX

then the strongly typed languages C# or Java will be used.

The project will then have access to the above engine/framework's libraries and built-in APIs for use. The linter tools, code-coverage, and unit-test modules will once again vary depending on the engine. For example, if the libGDX framework is is used, then the project can use the combination of JUnit (unit testing), JaCoCo (code coverage) and Spotbugs+Checkstyle (linting) for continuous integration/development (CI/CD). As for Godot, GdUnit4, GdUnit4 coverage, and GdLint are available and so on so forth with the others.

The project will utilize GitHub Actions for CI/CD after installation of the code testing modules to ensure proper bugs and errors are caught immediately. A team-member will have the luxury to work on a sub-branch and perform pull requests and pushes without having to worry about breaking another teammates codebase. This will trigger GitHub actions to set-up the framework environment, build the project, and perform all specified unit tests and code coverage. If the push or pull request fails one of the code coverage or unit tests, then the push is rejected. Git and GitHub projects will be used for proper version-control of the code and ease of access via the cloud storage. The team will then implement the roguelike functionality instead of using previous implements (if any) as well as the main gameplay loop in order to have full control and familiarity over manipulating game mechanics, game variables and other processes.

11 Use of AI and LLMs

Throughout this project, the team will use LLMs (Large Language Models) as needed to find/confirm common code syntax, grammatical/spelling rules, and other minor clarifications. LLMs may also be used to conduct brief 'sanity checks' of documentation and source code. Finally, if determined to be beneficial and appropriate by a team member, LLMs may be used to generate source code, primarily for small sections and methods (such as implementing a set of functions to request an object's basic, primary attributes). The use of LLMs to generate large sections of source code will be avoided, and must be discussed with the team if a team member intends to use an LLM for this purpose. LLMs will not be used to generate documentation text, beyond simple automations or grammatical edits that do not significantly change a document's presented ideas (i.e. creating an itemized list of text that was already written by a team member, vs. generating an entire document section).

Any work products (documentation, source code) generated by LLMs must be reviewed and verified for correctness by the team member using the LLM, and the use of the LLM must be transparently disclosed in each work product where it was used. Team members will not use any work products generated by LLMs that they cannot explain themselves or fully understand.

A variety of common free LLMs may be used throughout the project, such as OpenAI's ChatGPT, Anthropic's Claude, Google Gemini, or Microsoft Copilot. Other premium versions of these tools may be used if team members decide to purchase personal access.

12 Coding Standard

The coding standard used throughout the project will depend on the game engine or framework used to develop the game:

- If Godot is selected, the Godot documentation's GDScript style guide [1] will be used as a standard. If Godot's option for writing code in C# is used, the Godot documentation's C# style guide [2] will be used, to follow an appropriate style for C# development in a video game context.
- If GameMaker is used, the GameMaker Manual's best practices page [3] will be used as a standard. GameMaker Language currently does not have an official style guide beyond this document, so the engine's best practice documentation will be used as an alternative.
- If LibGDX is used, Google's Java style guide [4] will be used as a coding standard.
- If Unity is used, Google's C# style guide [5] will be used as a coding standard.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

- OpenAI GPT-5.6 Sol
- OpenAI GPT-5.6 Luna
- Google Search AI Overview (Google Gemini)

13.2 How and Where Used

AI tools were used for assistance with LaTeX syntax and commands, during the development of the Workflow Plan, Project Decomposition and Scheduling, Use of AI and LLMs, and Coding Standard sections. AI was used in Team Roles to help brainstorm a list of roles to choose from. AI tools were not used to generate any text.

13.3 Verification of AI-Generated Content

Any LaTeX syntax provided by AI tools was verified by testing compilation in LaTeX documents, and testing the resulting output. Text was not generated by AI tools, and therefore no AI-generated text was required to be verified.

Appendix — Reflection

Reflection Questions Answered:

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Andreas:

1. A development plan is crucial to decide on various project details as a team before work on the project begins. This helps to ensure that all team members and project supervisors understand the specifics of project planning (Git workflows, meeting planning, etc.), to avoid any development risk incurred by delaying or failing to note important project details.
2. CI/CD pipelines offer many advantages through automation, by automating portions of many crucial development activities such as code linting, unit testing, and pull request review when team members push their code. A disadvantage of utilizing extensive CI/CD pipelines could be the overhead work required to create and maintain the pipelines, so a balance should be struck to ensure any CI/CD used assists development work as much as possible, without requiring any high-effort maintenance.
3. Our team had some small disagreements in different sections of the document and aspects of the development plan, such as concerning project planning / project workflow decisions, proof of concept ideas, and others. As with any team disconnect on project decisions, our team reviewed the decisions to be made together, collaboratively discussed the pros and cons of each option, and reached a conclusion as a team.

Owen:

1. Having a development plan before starting the project is crucial to really map out where the project will go. This is especially important in a team of people, since keeping everyone on the same page and planning logistics is the highest priority, and much different to a solo project, or one of a lower scope. Having a development plan, for the above reasons and more, help to avoid typical pitfalls of development like scope creep or a bad workflow.
2. CI/CD pipelines are extremely advantageous to use for a project, since it can help automate portions of the development cycle and make iterations easier. The initial setup of the pipelines and the potential upkeep are both

potential disadvantages since they would take more time, but the opposite argument could also be made. This argument is that if enough care and effort is put into creating the pipelines, then through the setup process the team would grow to understand their design process and goals more during this process.

3. In this deliverable our team disagreed on some small aspects, like idea specifics, wording, or design decisions. Resolutions were quite straightforward, since everyone communicates quite well so far. This makes it easy to review and discuss decisions while ensuring everyone stays informed, since discussions were civil and productive, reaching verdicts where everyone has had input.

Humna:

1. A development plan is crucial prior to starting the project as it allows for overall team organization and management, logistical details, expected technologies, workflow planning, and proof of concept planning. All of these factors are important to discuss as a group before starting the implementation stage to ensure that the team members are on the same page. Through this deliverable, our team specified a meeting schedule, the technologies we expect to use, how we will collaborate through our workflow plan, and our understanding of the proof of concept. As a team, we also set expectations on AI usage and held open discussions on our expectations.

2. There are many advantages of using CI/CD. It can allow for smoother progression as we develop our software by automatically checking changes and helping identify issues before they are merged into the main branch. It can also help make collaboration between team members more organized. However, one limitation to this approach is that it can take more time to set up and maintain, and workflow issues can take time to troubleshoot.

3. There were no major disagreements during the deliverable. Our group shared a similar vision of what we wanted to achieve from this project. Conversations were held regarding the team's openness and willingness to explore new technologies, and because we are in the same program, we generally use technologies such as GitHub and Git similarly.

Christopher:

1. A development plan is essential for any project to succeed, as it allows team members to be coordinated in their efforts and have a common understanding of how they want this project to move forward. They are very helpful in providing a guideline for the future of how we want things to be done and ensure that when we start our project, we are working with good principles and planning so we do not have to return later and fix incorrect or messy work. Without a development plan, progress would

be much more disorganized between team members, resulting in slower development with the end quality being significantly reduced.

2. CI/CD pipelines are helpful in the development process, allowing for automated processes to be run on code as it is developed, and can quickly be verified. Having code verified as it is developed will reduce technical debt, catching bugs early and preventing the need to revisit old code to debug. These changes speed up the development process significantly, giving developers instant feedback and reducing their workload.

3. Some disagreements occurred during this deliverable, but they were all quickly resolved. Any disagreement would be posted as a comment under the corresponding issue, and we would then come to a compromise. We also held a meeting to iron out larger disagreements and review documents as a team to ensure we agree with all information written down.

Andy:

1. It is important to implement a plan early on because it helps with thinking about the future actions that would otherwise be looked over. It also helps as a good document to look back on in case there are some things the team forgets about during development of the project.

2. The main benefit is the insurance that proper bug fixes and coding standards will be applied consistently throughout the whole project timeline. The main drawback is the large amount of overhead it takes to correctly implement the process.

3. There were several disagreements about the Proof of Concept; what needs to be added in the demo process, how narrow is PoC etc. There was constant debate on the technologies used as well; mainly the conversation on which engine/framework that the team will move on with.

Appendix — Team Charter

External Goals

The team's goal for this project is to aim for very high marks in this class and to produce a project we can all be proud to present at the end of the year, possibly continuing development after if desired. The group will do their best jobs possible to win a prize at the Capstone EXPO, but not solely commited to just winning as there are other objectives. The project should be worked on in such as way as to be presentable in each member's portfolio to showcase their experience.

Attendance

Expectations

Members should attend all meetings if possible, but if they need to miss a meeting, arrive later, or leave early, it is acceptable as long as they notify the team prior and provide any necessary details pertaining to the meeting.

Acceptable Excuse

Having a class during the time of the meeting, having a meeting arranged earlier with another team, needing to complete work with a sooner deadline than our project, and personal affairs that meet a certain level of seriousness are all acceptable excuses to miss meetings as long as all their work is still completed. Excuses which are not acceptable include working on less urgent classwork, because they simply don't think it is important or forgetting about the meeting.

In Case of Emergency

The team member should notify the rest of the team as soon as they can that something is going on that is hindering them from doing work or attending a meeting. If possible, they should also send all relevant information they have to the team relating to the task they are no longer able to complete. We understand that emergencies happen that can stop members from completing tasks, but as long as they take reasonable steps to allow other team members to carry on without them and split up their work easily, the team should be able to manage without that team member for the period of their emergency.

Accountability and Teamwork

Quality

Team members should be aware of upcoming deliverables and their assigned responsibilities, and come to meetings ready for open discussions. Team members are encouraged to come prepared with questions or points of confusion so they can be discussed and resolved together.

Deliverables should meet the requirements and be high quality, with no grammatical errors and accurate information that has been shared with the group.

Attitude

Team members should bring a positive attitude to all things work-related and be generally accepting and open-minded towards other team members and their ideas. We should not need a code of conduct and just use common sense to be respectful people to each other. Team members should only be harsh on each other if they are not meeting the team's expectations in any way.

Stay on Track

Weekly recurring team meetings will be scheduled to ensure the team stays on track. During these meetings, team members will share their updates and set goals for the upcoming week. By doing so, all members will be aware of the project's progression, the collective goals of the team, and the next steps being worked towards.

Team members performing well will be recognized through the weekly meetings, as each member's contributions will be reviewed to ensure the group's quality of work is maintained. For team members who are underperforming, the group will hold open conversations to assess how the team can operate differently. This may be identified through repeated missed meetings and incomplete assigned responsibilities. If the behaviour continues, the team will discuss the issue with the TA.

Team Building

The team will not have any specific team-building activities, but will be expected to bond simply through working together on a project we enjoy.

Decision Making

Decisions the team makes should mostly be compromises of all team members' opinions and ideas until we can come to a consensus that we all agree with the idea. If it is not possible to compromise or come to a consensus, we will have to vote on all the ideas we have presented so far.

References

[1] Godot Engine, "GDScript style guide," *Godot Engine Documentation*, 2026, https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_styleguide.html, (accessed Sep. 25, 2026).

[2] ——, "C# style guide," *Godot Engine Documentation*, 2026, https://docs.godotengine.org/en/stable/tutorials/scripting/c_sharp/c_sharp_style_guide.html, (accessed Sep. 25, 2026).

[3] YoYo Games, "Best Practices When Programming," *GameMaker Manual*, 2026, https://manual.gamemaker.io/lts/en/index.htm#t=Additional_Information%2FBest_Practices_When_Programming.htm, (accessed Sep. 26, 2026).

[4] Google, "Google Java Style Guide," *GitHub*, 2026, https://google.github.io/styleguide/javaguide.html#s5-naming, (accessed Sep. 26, 2026).

[5] ——, "C# at Google Style Guide," *GitHub*, 2026, https://google.github.io/styleguide/csharp-style.html, (accessed Sep. 26, 2026).